

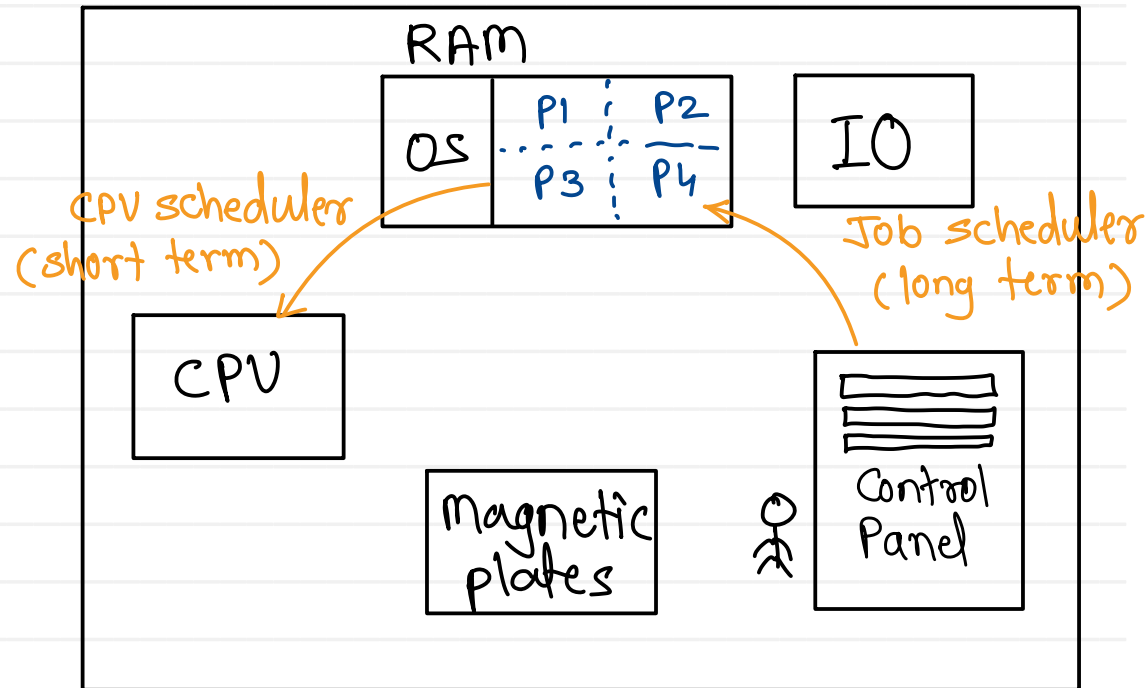


Sunbeam Institute of Information Technology
Pune and Karad

Module - Operating System Concepts and Linux Programming

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com



1. Resident Monitor
2. Batch System
3. Multi programming system

- multiple programs are loaded into RAM

CPU burst : time spent on CPU by process

IO burst : time spent on IO by process

$\text{CPU burst} > \text{IO burst}$: CPU bound process

$\text{IO burst} > \text{CPU burst}$: IO bound process.

- mixture of CPU & IO bound process was loaded into RAM

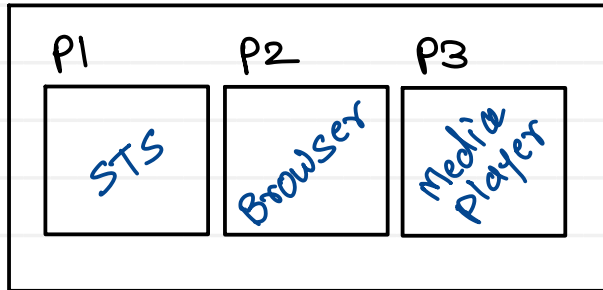
4. Time sharing system / Multitasking :

Response time < 1 sec

- CPU time is shared in all the processes of RAM

There are two types of multitasking.

i. Process based Multitasking



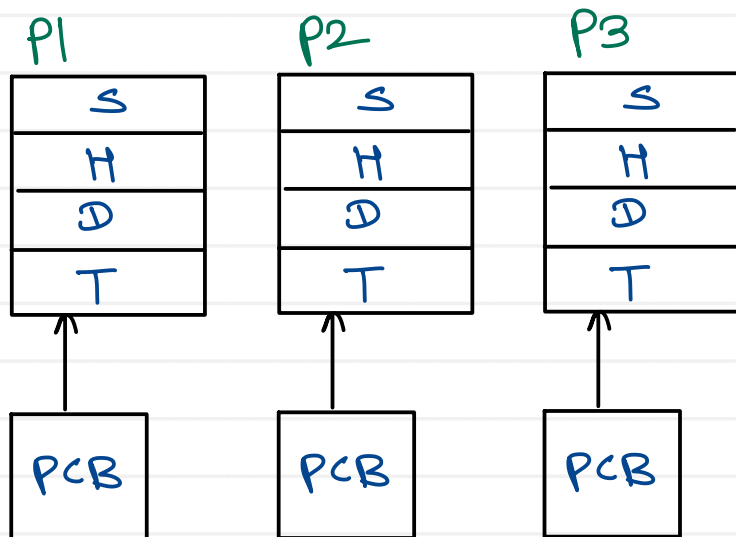
- system wide multitasking

ii. Thread based Multitasking
(Multi Threading)



- Multitasking within process

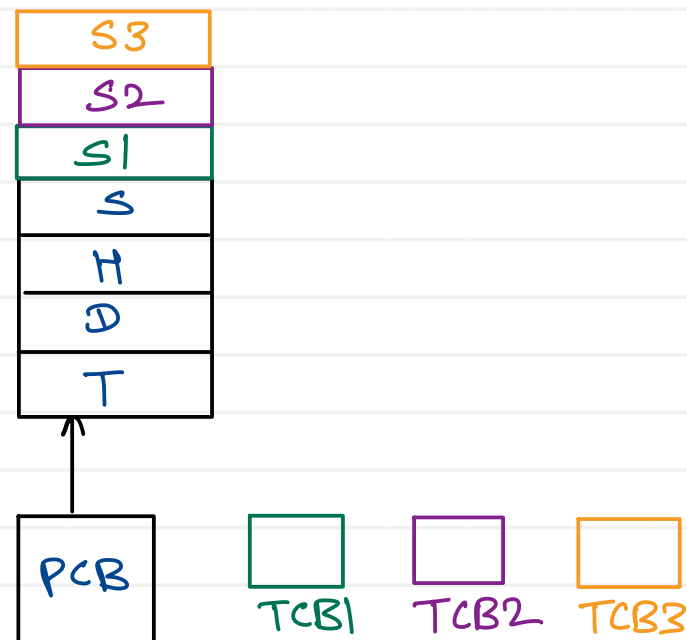
Processes are always created inside system



- Process is container of resources.
- Thread is alive entity which executes/run in container.
- every process has one default main thread.

Thread : light weight process

Threads are always created inside process.



5. Multiprocessing system:

- on single chip multiple CPUs are putted together, such process's are called as 'multi core' / 'multi processor'
- multiple processes can be executed parallelly.
- such systems are called as "multiprocessing systems".

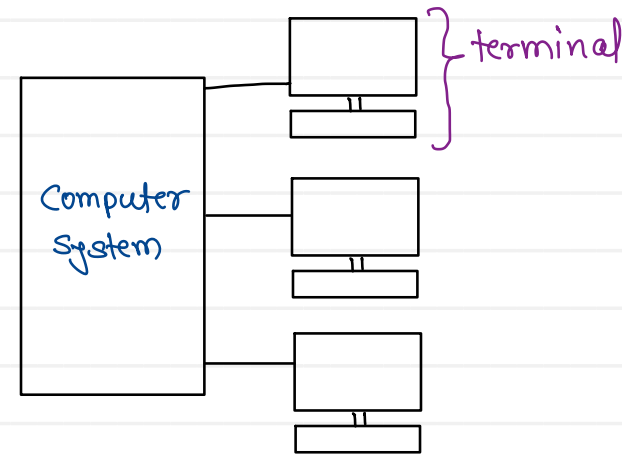
1. Symmetric Multiprocessing

OS treats all processor equally & schedules any process on any core (processor).

2. Asymmetric Multiprocessing

OS do not treat all cores equally. OS only scheduler process for one core & that core internally divides task into remaining cores.

6. Multiuser system



ps	uname	tz
top	uname -r	whoami
	uname -a	who
		N

deposit

Thread 1 () {

balance + = 2000 ;

- ① read balance
- ② update balance
- ③ write balance

}

~~10000~~ ~~8000~~
12000
balance

withdraw

Thread 2 () {

balance - = 2000 ;

- ① read balance
- ② update balance
- ③ write balance

}

deposit

Thread1() {

P(S)

balance + = 2000;

V(S)

}

- ① ① read balance
- ② ② update balance
- ③ ③ write balance

~~10000~~

balance

~~8000~~
12000

withdraw

Thread2() {

P(S)

balance - = 2000;

V(S)

}

- ① ① read balance
- ② ② update balance
- ③ ③ write balance

~~X0~~ -1

S

- internally it is a counter

① Dec op / PC) / wait()

- dec count

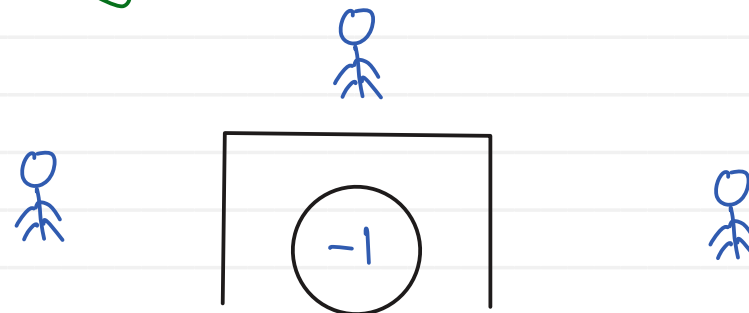
- if count < 0 , then block current process (thread)

② Inc op / VC) / post()

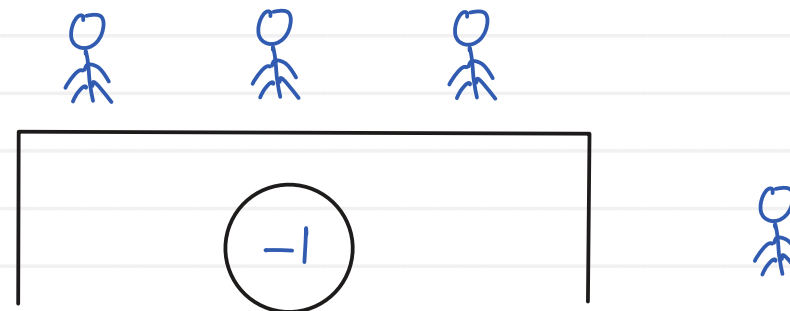
- inc count

- if processes are blocked on this semaphore, wakeup one.

① Binary semaphore



② Counting semaphore



- count no. of available resources

```
producer() {
```

```
while{
```

```
    P(e)
```

```
    P(b)
```

```
    ↓
```

```
    V(b)
```

```
    V(f)
```

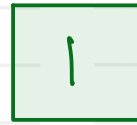
```
}
```

```
}
```

circular buffer



e



b



f

```
consumer() {
```

```
while{
```

```
    P(f)
```

```
    P(b)
```

```
    ⏪
```

```
    V(b)
```

```
    V(e)
```

```
}
```

```
}
```

Mutual Exclusion

(one at a time)

- mutex is same as binary semaphore
- operations : `lock()` - before using resource
`unlock()` - after releasing resource
- process which will lock the mutex will become owner of that mutex.
- only owner can unlock the mutex

- infinite waiting of processes for a resource
- deadlock will occur in system when all four conditions will be true at a time
 1. Mutual Exclusion
 2. No preemption
 3. Hold & wait
 4. Circular wait



Prevention

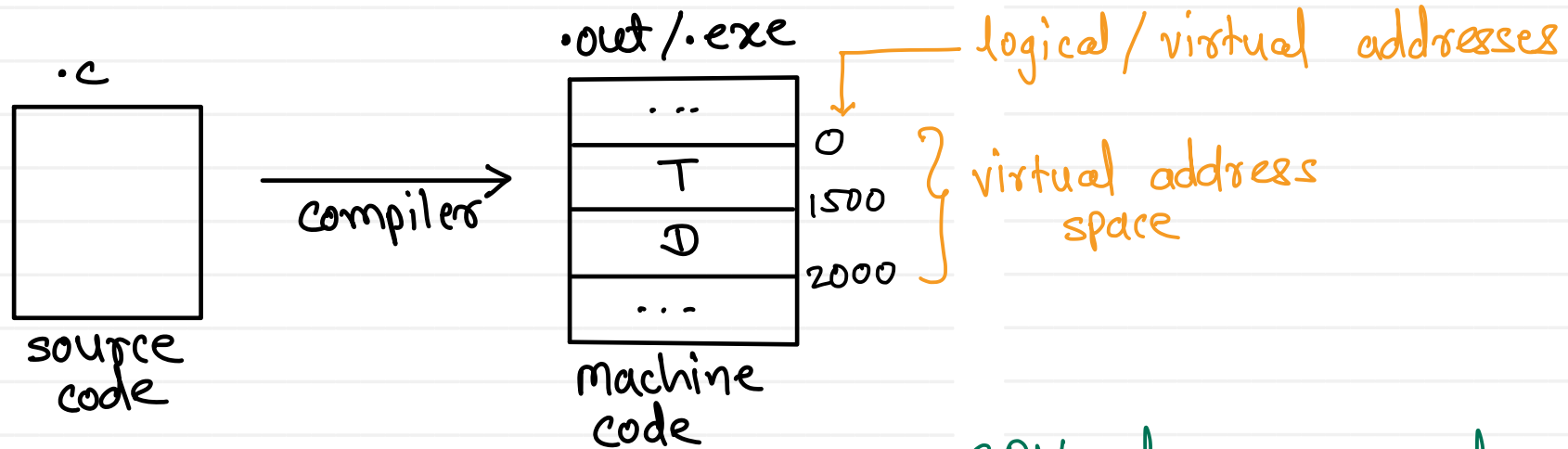
- while implementing OS, it is always ensured that one condition out of four will be always false

Avoidance

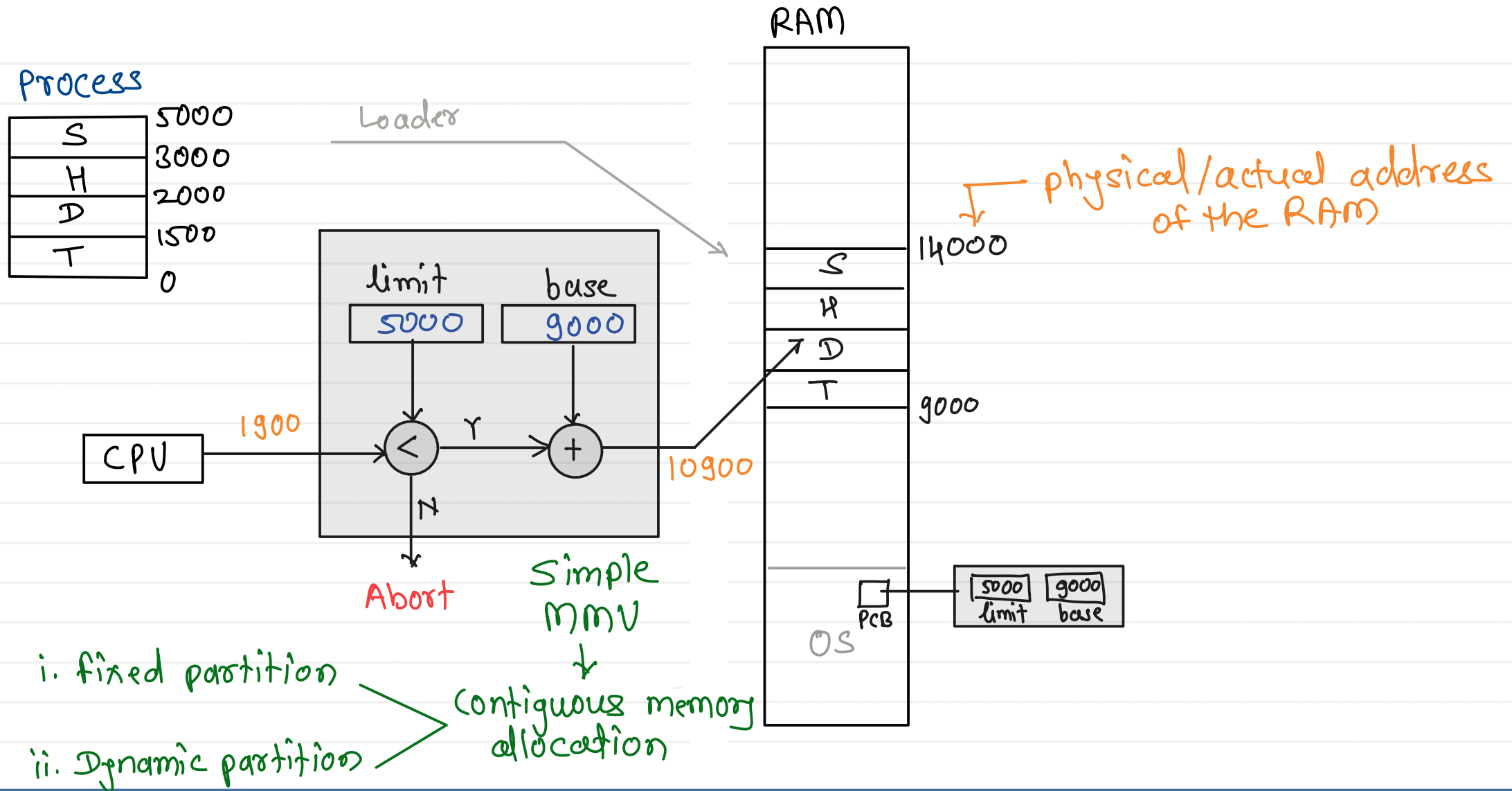
1. Resource allocation graph
2. Banker's algorithm

Recovery

- resource preemption
- forceful termination of process

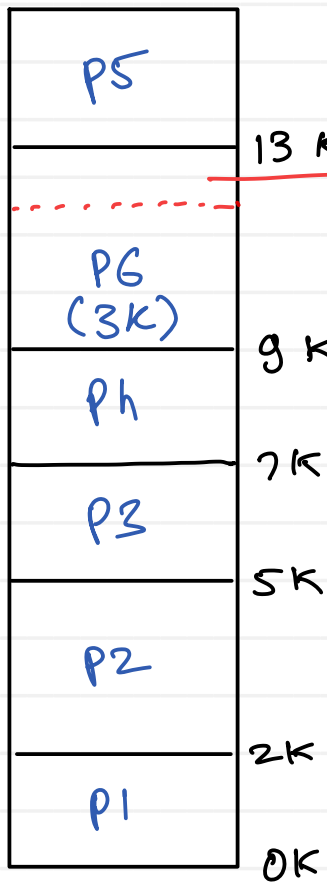


CPU always executes program in virtual address space.



Contiguous memory allocation

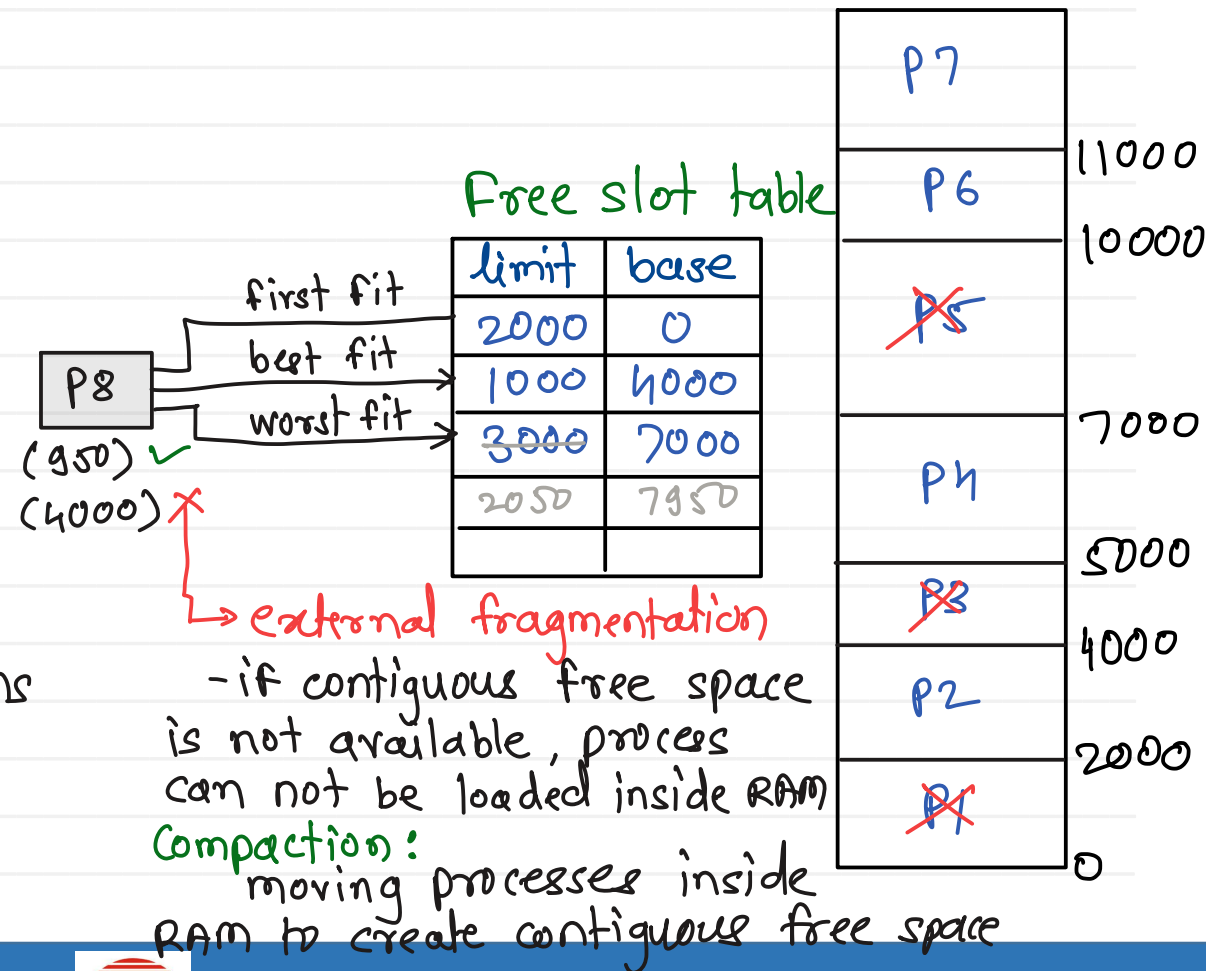
Fixed Partition



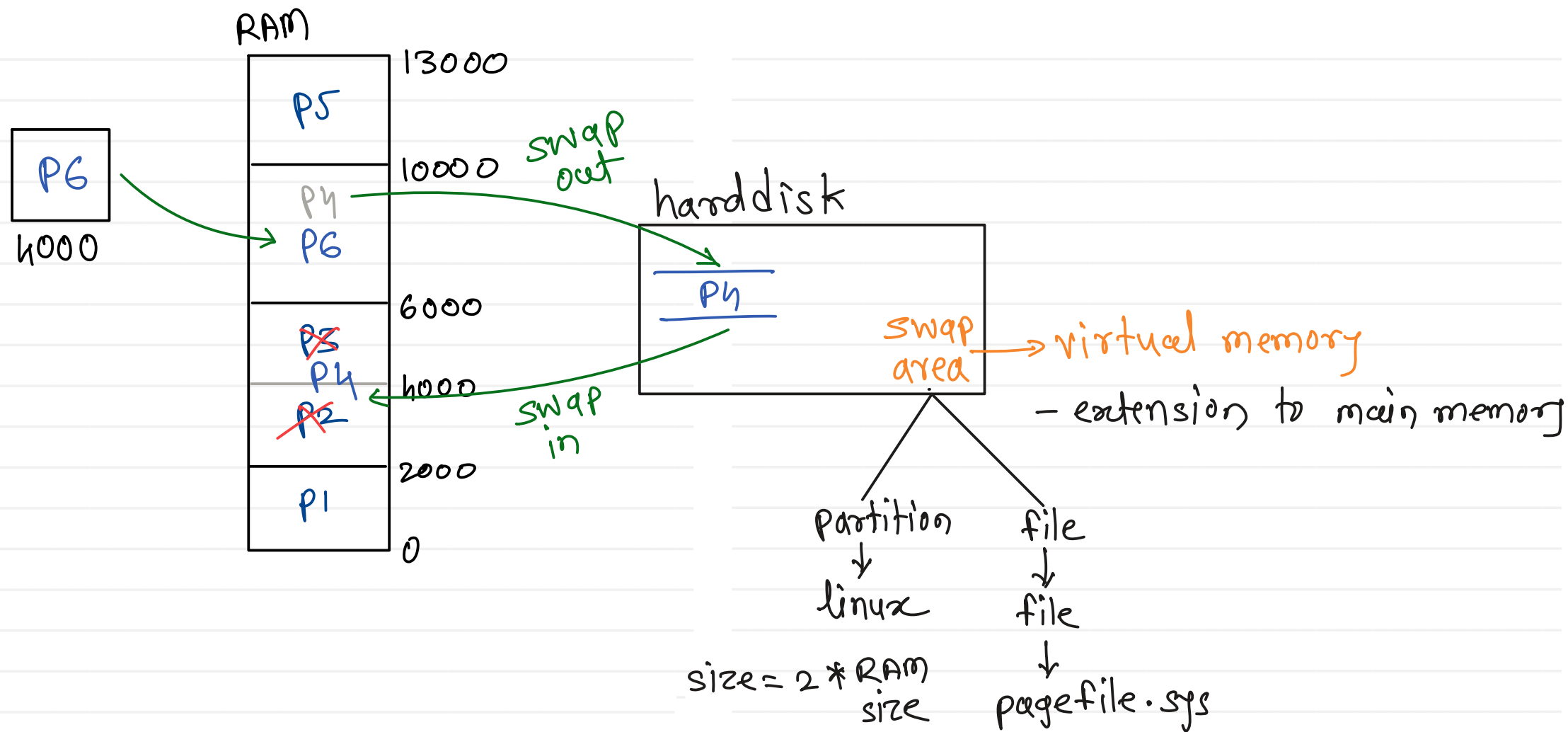
internal fragmentation
 - if process is not utilizing whole allocated partition, some part of it will be wasted

limitations:
 - no. of processes is fixed to number of partitions
 - max size of process is equal to size of max partition

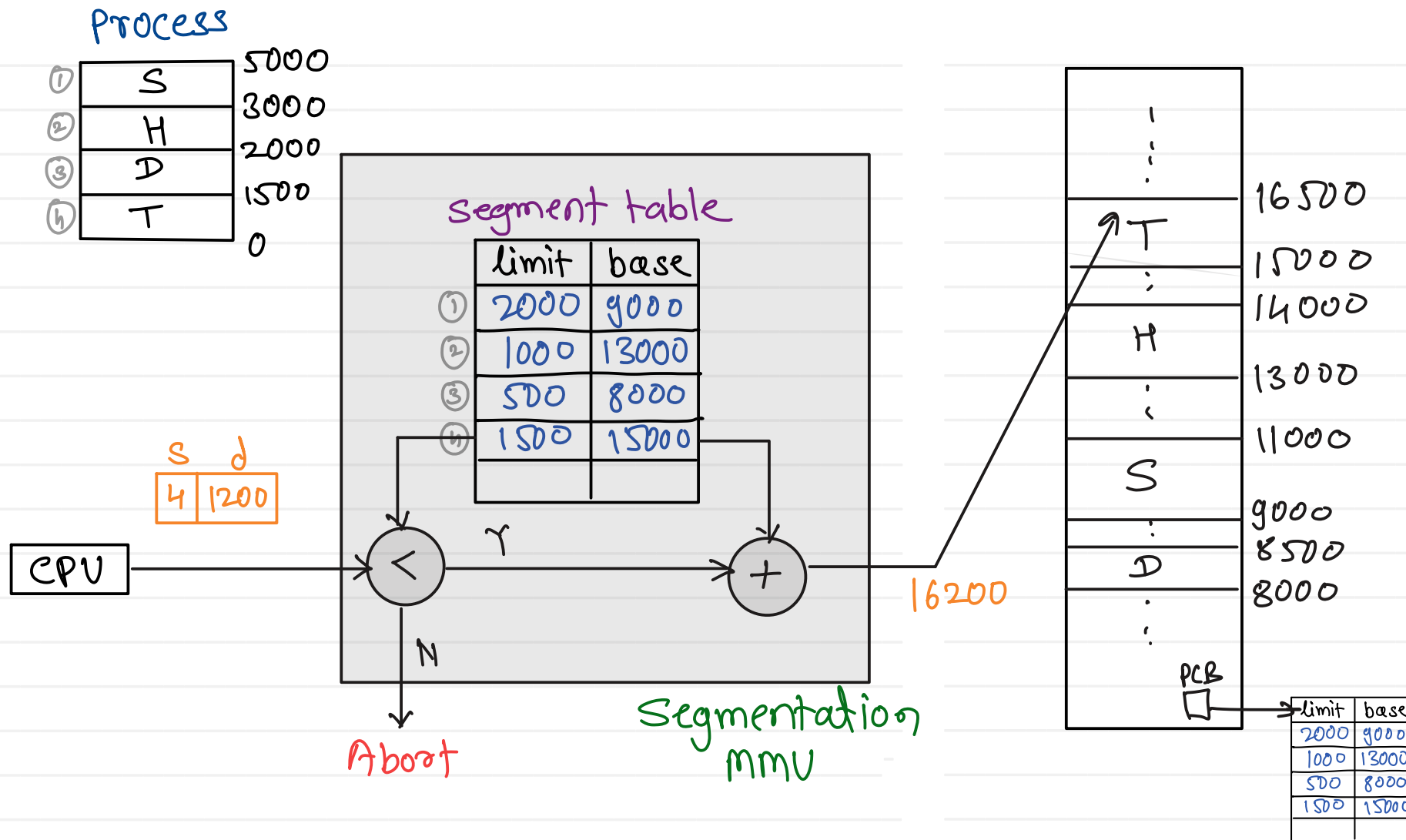
Dynamic Partition

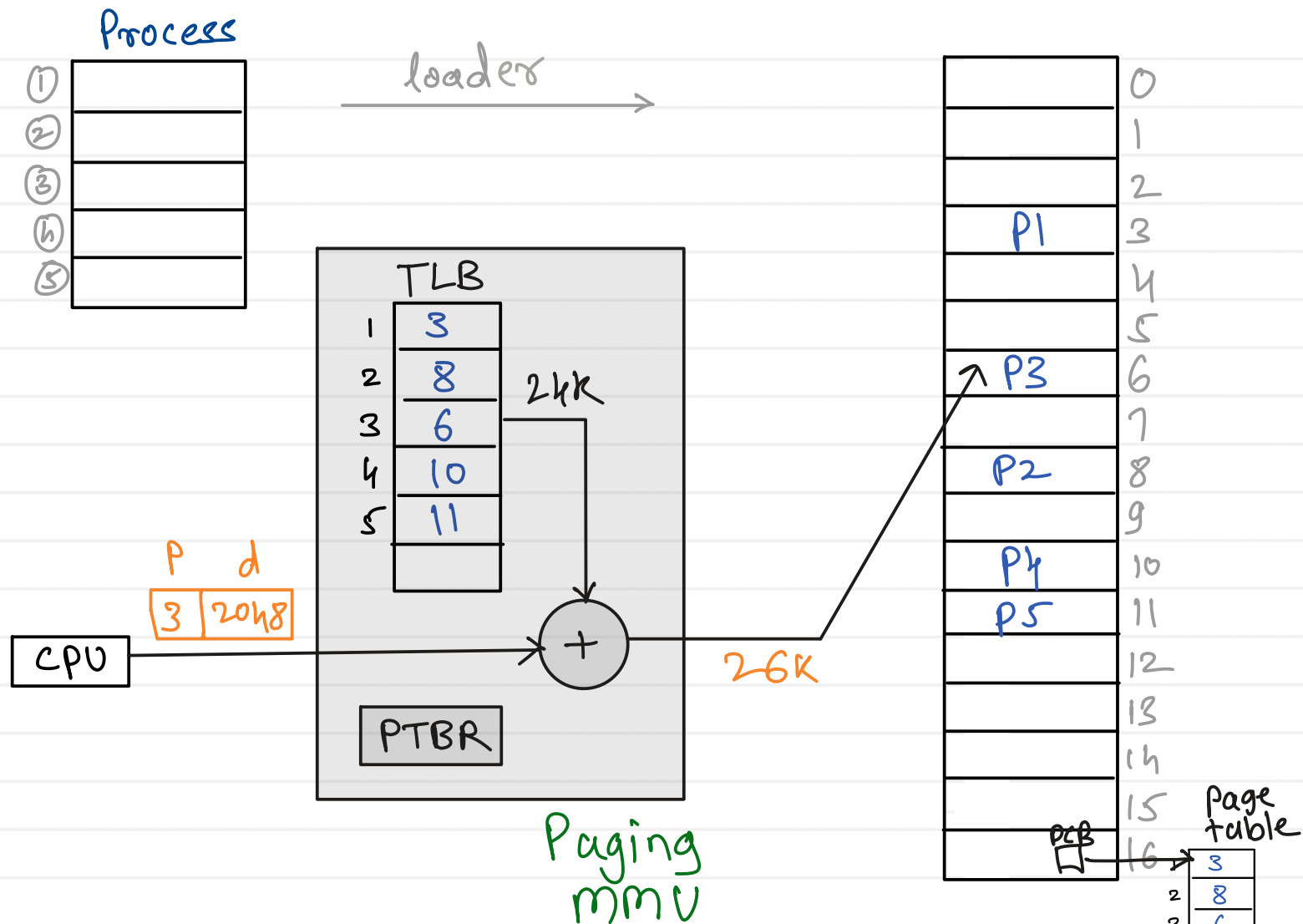


Virtual memory



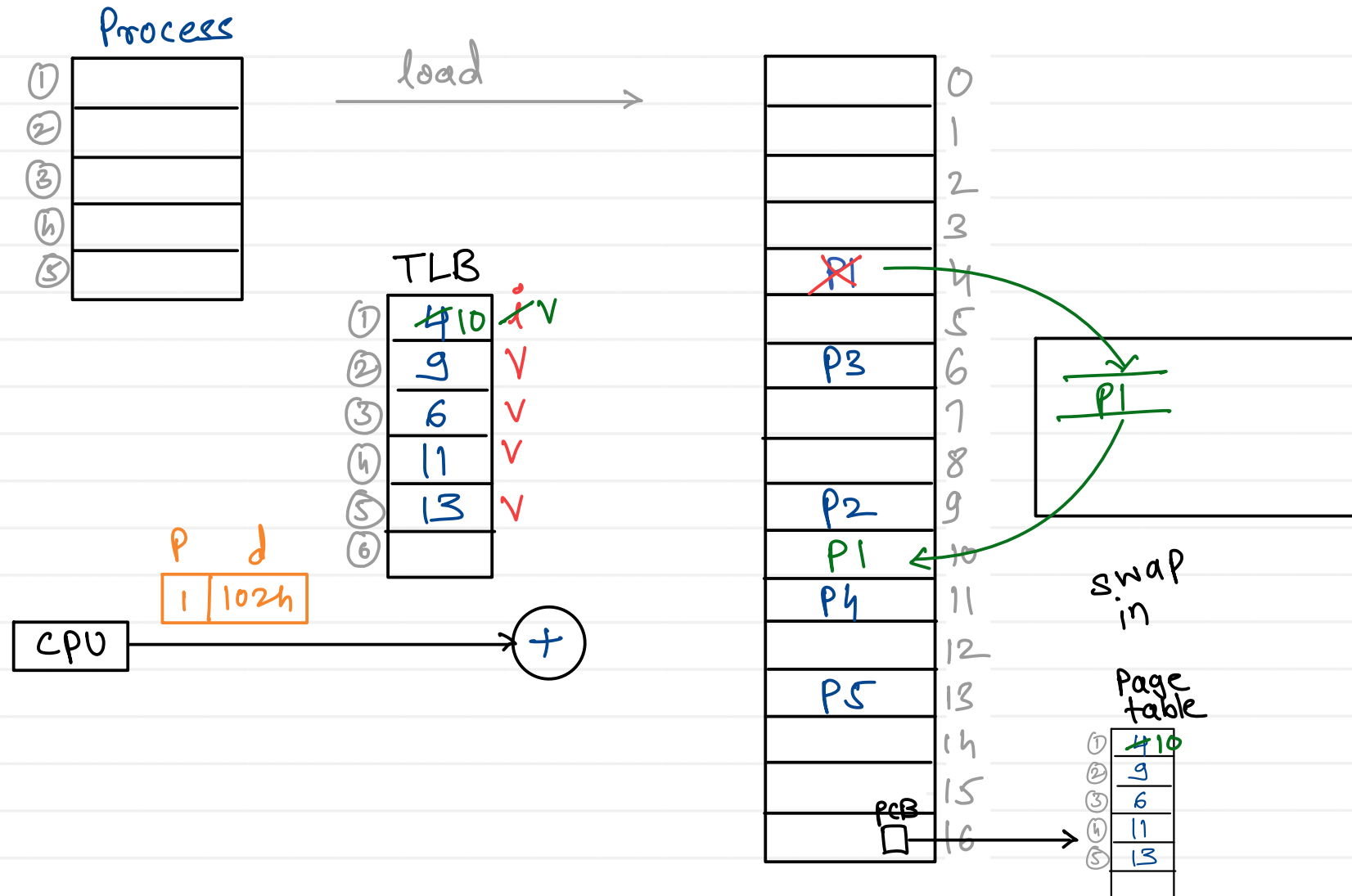
Segmentation MMU





- RAM is divided into fixed equal size partitions
 frame/physical page
 frame = 4096 bytes

- process is also divided into partitions of size equal to frame size
 page/logical page



- When CPU tries to access address of some invalid entry page, "page fault" is generated in system.

- Whenever page fault is generated, page fault handler is called.

- page fault handler will first validate the address, find the free frame, swap in the page, modify entry of page table